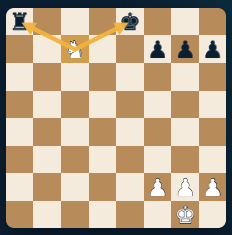
TURNING A CHESS ENGINE INTO SCIENCE AI

Chess AI is now solving science problems.

The search that lets a computer beat chess champions now helps scientists at **Brookhaven National Laboratory** decide which measurement to take next. In two weeks, you and a partner build that kind of AI yourselves in Python, starting with a chess engine named **Remus**, and then turn its thinking on a science puzzle.

10 days, 8:30 to 4:30 | Ages about 12 to 14 | No coding experience needed

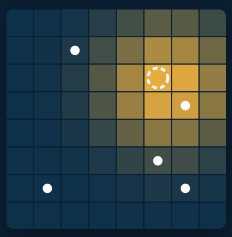


ON THE BOARD

Which move next?

Remus looks ahead and finds the knight fork.

SAME SEARCH: LOOK AHEAD, SCORE, CHOOSE



IN THE LAB

Which measurement next?

An AI picks the spot most likely to reveal the hidden peak.

What you and your partner will build

1

An engine that knows every rule

Castling, en passant, promoting to a knight, check, checkmate, stalemate, and draws by repetition.

2

A player inside a real chess app

By Day 7 you play against your own engine in En Croissant, the same kind of app chess clubs use.

3

A thinker that looks ahead

Remus searches thousands of positions, scores them, and can tell you why it chose its move.

4

A tournament entry

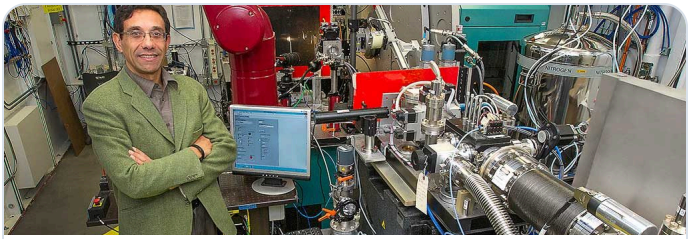
On Day 10 every team's engine plays every other team's engine in a round-robin tournament.

FUN FACT YOU WILL PROVE

Chess explodes fast.

Count every possible game from the starting position. Your code has to match these exact numbers, and on Day 6 it will.

After 1 move	20
After 2 moves	400
After 3 moves	8,902
After 4 moves	197,281



YOUR SCIENCE CHALLENGE

Find the hidden peak.

On Day 10 you program an AI that tracks down a hidden hot spot on a sample in as few measurements as possible. It is the same choice an AI agent makes on a real beamline at Brookhaven's NSLS-II: where to measure next.

NSLS-II beamline. Photo: Brookhaven National Laboratory

Not a chess-strength contest
A beginner who knows the moves is ready. The engine does the hard calculating.

Not a typing class
You and your partner trade turns: one types, one checks and steers.

Not a black box
Every line of Remus is plain Python you can read and explain.

WHAT THE TWO WEEKS LOOK LIKE

From your first line of code to your own AI in ten days.

Every day your engine can do something it could not do the day before: make a move, spot a check, see three moves ahead. Then you aim your AI at a science problem straight from Brookhaven's labs: choosing the measurement that reveals what a sample is hiding. By the end you can explain every line of an AI you built yourself.

Every day TEN WEEKDAYS IN A ROW

8:30 to 10:00	10:00	10:30 to 12:00	12:00 to 1:00	1:00 to 2:30	2:30	3:00 to 4:30
Period 1	Break	Period 2	Lunch	Period 3	Break	Period 4

How a period runs: a hands-on puzzle with paper and cards, a small Python lab to practice the idea, then you write a real Remus function and run its tests until they pass.

Ten days, one engine EACH DAY ADDS A PIECE

DAY 1
Setup and the board
Your laptop, VS Code, and Python. Draw a chessboard in code.

DAY 2
Functions and records
Teach Remus to make a move and take it back exactly.

DAY 3
Loops that walk the board
Knights, kings, sliding rooks and bishops, and pawns.

DAY 4
Legal moves
Is the king attacked? Throw away every move that leaves it in check.

DAY 5
Special moves
Castling, en passant, and promoting a pawn to any piece.

DAY 6
Counting the tree
Count every possible game and match the official numbers.

DAY 7
A whole game, in a real app
Notation, draws, and Remus playing inside En Croissant.

DAY 8
Judging a position
Material, piece placement, and looking moves ahead.

DAY 9
Searching smarter
Alpha-beta pruning: same move, a fraction of the work.

DAY 10
Science AI and tournament
Aim your AI at a science problem, then a round-robin tournament.

What you take home

1. **Your engine on a USB drive**, with its full history showing every change you and your partner made.
2. **Your tournament games**, saved so you can replay every move.
3. **Your team name in the credits** of the class engine for each function you wrote.
4. **Real Python skills**: functions, loops, lists, recursion, testing, and version control.

For parents

Does my child need a laptop?
No. Each pair uses a class laptop with everything already installed.

Will my child need online accounts?
No. There are no sign-ups of any kind. All work stays on the class laptops and goes home on USB.

What if my child has never coded?
That is expected. Python is taught from the first line, with labs before every engine task.

Who is in the room?
Twenty students in ten pairs, with an instructor and an assistant, so help is never far away.

You are ready if you

- ✔ know how each chess piece moves
- ✔ like solving puzzles with a partner
- ✔ are curious how computers think
- ✔ want to learn about AI